Enterprise Coding Agent — Architecture & Implementation Plan

Version 3 — agentic-only, Electron + Rust, Qwen 3.6. Last updated: 2026-04-28

1. Executive Summary

We're building an internal agentic coding assistant: a desktop app a developer launches when they want to hand off a real task — fix a failing test, add a feature, refactor a module — and a self-hosted Qwen 3.6 model on a single GPU server inside the company perimeter does the work end-to-end. Code never leaves the network.

The product is **not** a tab-completion engine and **not** an IDE plugin. It's a focused agent surface where:

1. The developer opens a project, asks for a change in plain English.
2. The agent reads the repo, runs commands, sets breakpoints, makes edits.
3. The developer reviews the resulting diff in the app, comments on specific lines, and either applies it or sends comments back for revision.
4. The cycle iterates until the developer hits "Apply".

Three architectural pillars support this:

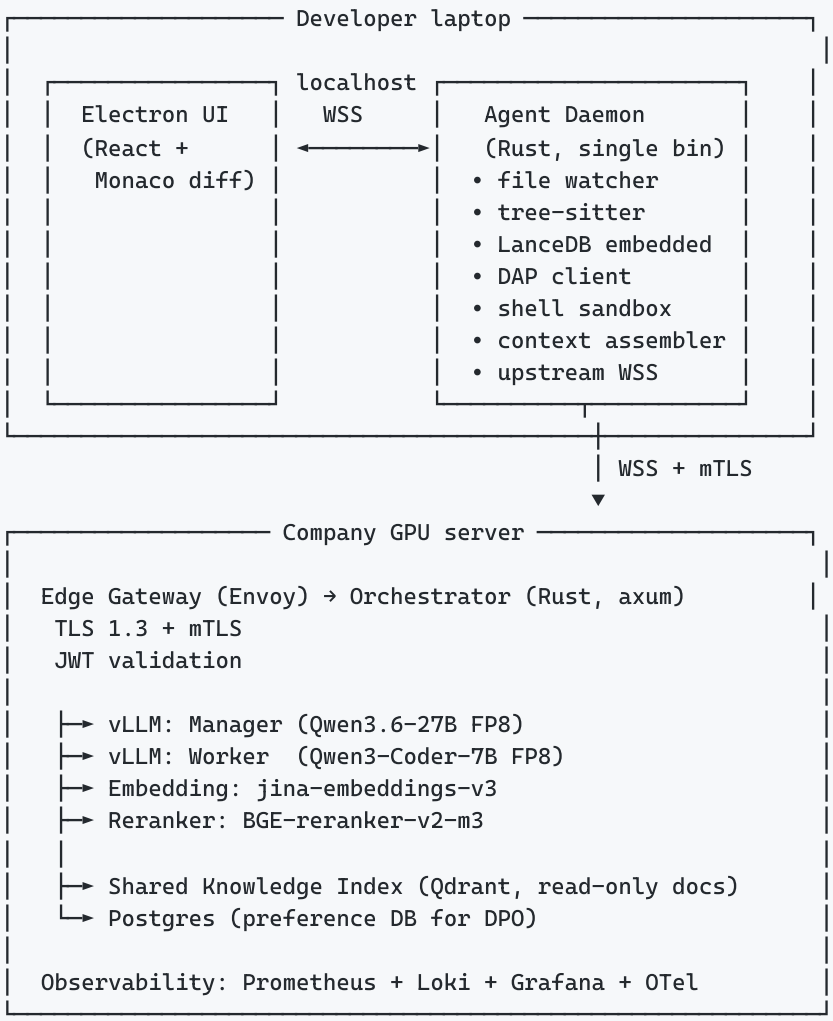
- **Manager-Worker LLM topology.** Qwen3.6-27B (dense, FP8) is the Manager — it plans and synthesizes. A Qwen3-Coder-7B Worker handles bulk reading and summarization so the Manager's context stays packed with synthesis instead of raw bytes.
- **Project-scoped retrieval, invisible to the user.** Each project the developer adds gets its own embedded LanceDB inside the app. The user never sees it; they just "Add Project" and ask questions.
- **Tools as ground truth.** The agent can run a shell, query the LSP, and — new in this plan — drive a real debugger via DAP. When unsure, it sets a breakpoint and inspects values rather than guessing.

Continuous improvement comes from a quarterly DPO fine-tune on the developer's own review signal: every accepted hunk, rejected hunk, manual edit, and inline comment is preference data.

2. Design Principles

Principle	Consequence
Agentic, not autocomplete	Optimize for multi-turn task completion, not for low-latency single-token output. No FIM / tab paths.
Information density	Context is constructed in named, budgeted layers; nothing arrives by accident.
Project-scoped, transparent memory	One vector DB per project. The user never thinks about it.
Privacy-by-design	Source files, the vector DB, debugger sessions, and tool output stay on the laptop. Only request-time prompt layers cross the wire.
Tools as ground truth	Anything verifiable (versions, tests, runtime values) is verified, not hallucinated.
Review-first UX	Diffs always go through an in-app review surface. Comments on specific lines feed the next turn.
Deterministic prompt assembly	Same inputs → same prompt. Required for prefix caching and for reproducing failures.
Fail-soft client	The Rust agent daemon survives UI crashes; reconnects survive WS drops; partial answers stream.

3. System Topology



4. Communication Layer

4.1 Why WebSocket (not gRPC, not raw HTTP)

Same reasoning as before: bidirectional streaming, multiplexed via per-request IDs, traverses corporate proxies as TLS 443, debuggable in Chrome DevTools, native fit for Electron. gRPC is faster on paper but harder to operate through corporate TLS-inspecting proxies and harder to debug.

4.2 Wire Protocol

- **Transport:** WSS (TLS 1.3) on port 443, with **mTLS** for client identity.
- **Framing:** JSON-lines for control; binary `MessagePack` opt-in for embeddings and large file payloads, negotiated at handshake (`v1.json` vs `v1.msgpack`).

- **Heartbeat:** client `ping` every 25 s; connection considered dead after 60 s of silence.
- **Reconnect:** exponential backoff (1, 2, 4, 8, 16, 30 s capped) with ±20% jitter. Each session has a `resume_token` so in-flight streams resume from the last-acknowledged sequence.

4.3 Message Schema

```
{
  "v": 1,
  "id": "01HXY...",         // ULID per logical request
  "session": "sess_abc123",
  "seq": 42,                // monotonic per (session, id)
  "type": "request"
        | "stream"          // partial token output
        | "tool_call"       // server asks client to execute a tool
        | "tool_result"     // client returns the tool output
        | "tool_event"      // async push from a long-lived tool (e.g. debugger stopped)
        | "review_state"    // user's review comments + manual edits, sent back
        | "done"
        | "error"
        | "ping" | "pong",
  "payload": { ... }
}
```

Concrete agentic exchange ("fix the failing test in `auth/session_test.py`"):

```
C → S  request    { task:"fix_failing_test", file:"auth/session_test.py",
                    context:{...layers L0–L9...} }
S → C  stream     { delta:"Reading the test ..." }
S → C  tool_call  { tool:"shell.exec", args:{cmd:"pytest auth/session_test.py -x"} }
C → S  tool_result{ exit:1, stdout:"...", stderr:"AssertionError: expected 200, got 401" }
S → C  tool_call  { tool:"debug.start", args:{lang:"python", program:"pytest", ...} }
S → C  tool_call  { tool:"debug.set_breakpoint", args:{file:"auth/session.py", line:84} }
S → C  tool_call  { tool:"debug.continue", args:{} }
C → S  tool_event { event:"stopped", reason:"breakpoint", frame:84, vars:{...} }
S → C  tool_call  { tool:"debug.evaluate", args:{expr:"self._cached_token"} }
C → S  tool_result{ value:"None" }
S → C  stream     { delta:"Found it. The cache is being invalidated ..." }
S → C  tool_call  { tool:"fs.write", args:{file:"auth/session.py",
                    patch:"@@ -82,7 +82,9 ...", dry_run:true} }
S → C  done       { proposal_id:"prop_xyz", usage:{...} }
```

The user sees the diff in the app, may comment, and either applies or iterates (§10).

4.4 Authentication

1. Developer SSO via OIDC → short-lived JWT (15 min).
2. WebSocket upgrade carries the JWT in `Authorization: Bearer …` plus an mTLS cert issued from a per-laptop CA at install time.
3. Edge gateway validates JWT signature, audience, mTLS chain, source IP allowlist.

4. Token refresh via `POST /auth/refresh`. Active sockets are not re-authed mid-stream.

4.5 Backpressure & Flow Control

- Per-session in-flight cap: 4 concurrent requests.
- Server emits `type:"queued"` if the request waits more than 200 ms; UI surfaces "waiting for capacity".
- Stream chunks are flushed to the UI on a 16 ms timer.

---

5. Server-Side Model Serving

5.1 Hardware Options — picking the GPU tier

For 10 agentic developers and a 27B Manager, datacenter-class hardware (H100/H200) is overprovisioned. The honest market in April 2026 looks like this:

GPU	VRAM	$/card (street)	Mem bandwidth	NVLink	Fits Qwen3.6-27B FP8 on one card?
H100 80GB SXM	80 GB	$35–40 K	3,350 GB/s	Yes	Yes, comfortable
H100 80GB PCIe	80 GB	$25–30 K	2,000 GB/s	Yes	Yes, comfortable
RTX PRO 6000 Blackwell	**96 GB**	**$8–9 K**	**1,792 GB/s**	**No**	**Yes, with huge headroom**
L40S	48 GB	$7–9 K	864 GB/s	No	Yes, tight on KV cache
RTX 5090	32 GB	$2–3 K	1,792 GB/s	No	**No** — 27 GB weights leave ~3 GB for KV

Two facts drive the choice:

- **Pro 6000 beats H100 SXM on this exact workload.** Published vLLM benchmarks show ~3,140 tok/s on the Pro 6000 vs ~2,987 tok/s on the H100 SXM for single-GPU FP8 inference — at one-third the hardware cost. The H100 only pulls ahead when you actually need multi-card tensor parallelism, which we don't with a model that fits on one card.
- **The 5090 is great $/GB but bad fit for our Manager.** 32 GB VRAM is below the 27 GB Manager weights + KV cache budget, so we'd need TP across 2–4 cards. NVLink is gone from consumer Blackwell, so TP runs over PCIe and pays a 10–15% throughput tax plus higher first-token latency. Then layer in 575 W per card, no ECC, and consumer warranties that vendors may void for sustained datacenter use.

Recommended configurations

🎯 Sweet spot — 1× RTX PRO 6000 Blackwell 96GB (~$10–12K all-in)

The Manager sits on a single card with absurd KV cache headroom (96 GB − 30 GB weights = 66 GB for KV → 16+ concurrent at 32 K context). Worker + Embed + Rerank co-resident: Worker 7 B FP8 (~~8 GB) + jina-v3 (~~2 GB) + BGE-reranker (~1 GB) = ~11 GB more. Total: ~52 GB used, ~44 GB free.

Fits in a Threadripper Pro / EPYC workstation chassis. No NVLink concerns because everything is on one card. **This is enough for 10 devs with comfortable headroom.**

Item	Spec	Cost
GPU	1× RTX PRO 6000 Blackwell 96GB	~$8,500
CPU	AMD Threadripper Pro 7965WX (24-core)	~$2,500
RAM	128 GB DDR5 ECC	~$800
Storage	2 TB NVMe + 4 TB SATA	~$400
Chassis + PSU + cooling	Workstation, 1600W PSU	~$1,200
Total		**~$13,400**

💪 Headroom for growth — 2× RTX PRO 6000 Blackwell (~$22K all-in)

Manager dedicated to GPU 0 (66 GB KV cache, 20+ concurrent), Worker + Embed + Rerank on GPU 1 with massive headroom. Comfortably handles 20–25 devs without a second box. Each model lives on one card, so the lack of NVLink is irrelevant.

💸 Aggressive budget — 4× RTX 5090 (~$15K all-in, with caveats)

Workable but operationally annoying:

- Manager needs TP=2 across 5090 #1 and #2 over PCIe (no NVLink) — ~10–15% slower than a single card of equivalent total VRAM, plus higher TTFT.
- Worker on 5090 #3.
- Embed + Rerank on 5090 #4.
- 4× 575 W = 2,300 W under load. Need a 240 V circuit and serious chassis cooling.
- No ECC; consumer warranties may not cover sustained datacenter use.
- Realistic concurrency on the Manager drops to ~4–6 simultaneous (vs 8+ on a Pro 6000), which is the floor for 10 devs at peak.

Item	Spec	Cost
GPUs	4× RTX 5090 32GB	~$10,000
Chassis + PSU (2200W+) + cooling	Server / heavy workstation	~$3,500
Rest of system	EPYC + 256GB RAM + storage	~$3,500
Total		**~$17,000**

Realistically only worth it if budget is the dominant constraint and someone is willing to operate it like a power user.

🏢 Datacenter / future-proof — 1× H100 80GB PCIe (~$33K all-in)

Pick this if (a) it has to live in a real server rack with redundancy, (b) you anticipate growing to 30+ devs and want NVLink-capable hardware to scale into, or (c) the company already has H100 capacity sitting around. Same 10-dev capacity as the Pro 6000 sweet-spot config, ~3× the price.

Recommendation

Start with 1× RTX PRO 6000 Blackwell 96GB. It's enough for 10 devs comfortably, costs ~$13K all-in, and the Pro 6000 has a clean upgrade path: when you need to support 20–25 devs, drop in a second card; for 50 devs, add a second box. If you eventually outgrow Pro 6000 economics (50+ devs), that's the point at which H100 NVLink starts paying for itself, and the model server gets re-architected anyway.

| CPU | AMD Threadripper Pro 7965WX (24-core) or EPYC 9354 | | RAM | 128 GB DDR5 ECC (256 GB if going to 2-card config) | | Storage | 2 TB NVMe (model weights, KV-cache spill, prefix cache) + 4 TB for logs/preference archive | | Network | 10 GbE intranet |

5.2 Inference Engine: vLLM

- **Continuous (rolling) batching** — new requests join the active batch the moment a slot frees up. ~5–10× throughput vs. naïve serving.
- **PagedAttention KV cache** — pages allocated like virtual memory; ~95% effective utilization of GPU memory for cache, which is what limits concurrency at long context.
- **Automatic prefix caching** — system prompt + tool schemas + repo card are identical across most requests. vLLM detects and reuses their KV cache. For our prompt shape this is a 30–50 % latency cut on repeat structures, and the win **gets bigger with more devs**, since shared traffic increases hit rate.
- **Multi-LoRA serving** — one base 27B in VRAM; we hot-swap per-team or per-task LoRA adapters at request time. This is how DPO fine-tunes ship without a redeploy.
- **MTP speculative decoding** (vLLM recipe for Qwen 3.6) — small predictor head emits 2 tokens/step, ratified by the main pass. ~1.5–2× decode throughput.

vLLM serve flags (production):

```
vllm serve Qwen/Qwen3.6-27B-FP8 \
  --tensor-parallel-size 1 \
  --max-model-len 65536 \
  --gpu-memory-utilization 0.92 \
  --enable-prefix-caching \
  --reasoning-parser qwen3 \
  --speculative-config '{"method":"mtp","num_speculative_tokens":2}' \
  --max-num-seqs 8
```

5.3 Manager Model — Qwen 3.6-27B

Why Qwen 3.6-27B and not 32B or 35B-A3B:

- **There is no Qwen 3.6-32B.** The Qwen 3.6 family ships as a dense **27B** and a sparse-MoE **35B-A3B** (35B total, 3B active per token, 256 experts with 9 active).
- The dense 27B is the right call for our workload: it scores **77.2 on SWE-bench Verified** and **59.3 on Terminal-Bench 2.0** — matching Claude 4.5 Opus exactly on the agentic terminal benchmark, and beating the larger 35B-A3B (51.5) on the same metric. On agentic coding tasks the dense 27B is the strongest open model in 2026.
- Dense 27B is also operationally simpler than the MoE: predictable memory, no router-imbalance pathologies, prefix caching benefits cleanly, and it fits comfortably on a single H100 at FP8.

Configuration:

- **Model:** `Qwen/Qwen3.6-27B-FP8`
- **Quant:** FP8 (~~30 GB weights). FP16 (~~54 GB) is also viable on a single H100; FP8 is preferred for the higher concurrency budget.
- **Tensor parallel:** 1 (fits on a single H100). TP=2 only if we need 128K+ context with high concurrency.
- **Context window:** configured to 64K (model supports more, but agentic turns rarely exceed 32K and capping the budget keeps KV cache usage predictable).
- **Reasoning:** Qwen 3.6's thinking mode is enabled and capped at 2K reasoning tokens before forced summarization. Use `--reasoning-parser qwen3`.
- **Sampling profiles** (orchestrator picks per task):
  - `plan` — temperature 0.2, top_p 0.9, max_tokens 1500
  - `synthesize` — temperature 0.4, top_p 0.9, max_tokens 4000
  - `creative` — temperature 0.7, top_p 0.95, max_tokens 4000

5.4 Worker Model — Qwen3-Coder-7B

Tier	Model	Quant	VRAM	GPU	Concurrency	TTFT target	Role
Manager	**Qwen3.6-27B**	FP8	~30 GB	0	8	<1.5 s	Plan, synthesize, drive tool loop, code-review reasoning
Worker	**Qwen3-Coder-7B**	FP8	~8 GB	1	16	<500 ms	Summarize files, compress tool output, log triage, distill diffs
Embed	jina-embeddings-v3	FP16	~2 GB	1	32	<50 ms	Vector embeddings for indexing & queries
Rerank	BGE-reranker-v2-m3	FP16	~1 GB	1	16	<100 ms	Cross-encode top-50 → top-10

GPU 1 hosts Worker + Embed + Rerank simultaneously (~11 GB used, plenty of headroom for KV cache on the 7B). One Manager, one small Worker — that's the whole stack.

Why no Heavy 14B / Fast 3B tiers: with the agentic-only scope, there's no tab-completion path that needs sub-200 ms latency, and the 27B Manager is fast enough that a 14B "intermediate" tier doesn't pay for itself. Two tiers (Manager for reasoning, Worker for compression) is the right amount of complexity for 10 devs.

5.5 Sizing & Linear Scaling

Workload profile (per developer, agentic):

- ~4–10 task starts/day during peak hours.
- A task = 5–15 turns. A turn = ~3–30 s of generation + tool calls in between.
- Active inference time per dev: ~5–15% of working hours.
- Average context per turn: 15–30 K tokens.

For 10 devs:

- Average concurrent active manager generations: **~2–4**.
- 95th-percentile peak: **~8 simultaneous**.
- Manager configured for **8 concurrent slots** at 32 K context → handled comfortably by a single RTX PRO 6000 Blackwell (96 GB) with the Worker + Embed + Rerank co-resident.
- Worker traffic is ~3× Manager traffic (every Manager turn triggers 2–4 worker compress/summarize calls). 16-concurrent on the 7 B is trivial.

Linear scaling — when it holds and when it doesn't:

Devs	Config	Approx. GPU cost	Notes
10	1× RTX Pro 6000 96GB	~$8.5 K	Baseline. Manager + Worker + Embed + Rerank all co-resident with headroom.
20–25	2× RTX Pro 6000 96GB	~$17 K	Manager on GPU 0 (huge KV budget), workers on GPU 1. Each model on one card, so no PCIe TP penalty.
50	4× RTX Pro 6000 across 2 boxes	~$34 K	Two replicas of the 2-card config, route by `hash(session_id)`. Prefix cache hit rate **improves** with more shared traffic.
100+	Mixed: H100 nodes for Manager, Pro 6000 for workers	$80 K+	Sub-linear above this point. Orchestrator becomes its own scaling unit; KV-cache eviction starts dominating. NVLink-capable hardware finally pays for itself when you need TP for higher per-Manager concurrency.

Why scaling is approximately linear in the 10–50 dev range:

1. The Manager dominates GPU spend, and Manager concurrency is bounded by KV-cache memory, which scales 1:1 with GPU memory.
2. Per-request compute is nearly identical across devs (same model, similar context shapes).
3. The vector DB does **not** scale with developer count — each dev's index lives on their own laptop.
4. The shared knowledge index sits in front of a small read-only Qdrant; query load is trivial (<10 QPS at 50 devs).

Where it breaks linearity:

- The orchestrator (single-node Rust process) handles ~5 K req/s comfortably; we're at <50 req/s at 50 devs, so this isn't a near-term problem.
- Network: 50 devs × ~2 MB/turn × 1 turn/min = ~100 MB/min — well below 25 GbE.
- Prefix cache: at very large scales, KV-cache eviction starts dominating; the fix is sharded routing so a project's traffic always hits the same node.

Cost rule of thumb: ~$1K of GPU capex per developer at the recommended Pro 6000 tier, sub-linear past 25 devs as cards saturate. Translated: +1 RTX Pro 6000 96GB per +12 devs, plus a fixed embedding/rerank slice that doesn't scale with developer count.

5.6 Routing Policy

A small set of rules in the orchestrator dispatches between Manager and Worker:

```
enum Tier { Manager, Worker }

fn route(task: &Task) -> Tier {
    match task.kind {
        TaskKind::Compress { tokens } if tokens > 1500 => Tier::Worker,
        TaskKind::Summarize    => Tier::Worker,
        TaskKind::ExtractSyms  => Tier::Worker,
        TaskKind::TriageLog    => Tier::Worker,
        TaskKind::Embed        => Tier::Worker, // bypassed to embedding service
        _                      => Tier::Manager,
    }
}
```

We start with rules. After ~3 months of trace data, we replace with a 100 M-param distillation trained on the trace.

---

6. Context Engineering Pipeline

6.1 Context Layers (Manager target ≈ 24 K tokens)

Layer	Owner	Budget	Source	Eviction
L0 — System	Server	600	Static prompt + capabilities	Never
L1 — Tool schemas	Server	1500	JSON schemas of allowed tools (now includes `debug.*`)	Filter to relevant tools per task
L2 — Project card	Daemon	800	Auto-generated digest (file tree, language stats, README excerpt, top-level entry points)	Refresh per session
L3 — Project memory	Daemon	1500	Auto-curated from past sessions (chosen patterns, recurring rejections); user-invisible	Sliding-window
L4 — Active focus	Daemon	2000	File the user pinned to the chat, or the file most recently mentioned	Re-read each turn
L5 — Git context	Daemon	500	Branch, last 3 commits, dirty diff summary	Re-read each turn
L6 — Retrieved chunks	Server (RAG)	6000	Top-K from vector DB, post-rerank	Per-turn
L7 — Worker briefs	Server	3000	Distilled output from worker for this query	Per-turn
L8 — History	Both	4000	Last N turns; older turns summarized by Worker	Sliding window + summary
L9 — Review state	Daemon	1500	**New.** User comments + manual edits since the last proposal (see §10)	Cleared on Apply
L10 — User query	Daemon	1000	Current message	n/a

Total ≈ 22 K. The remaining ~40 K of the 64 K budget is held for tool-result expansion mid-turn (a `git log` or `pytest` output that gets compressed by the Worker before reaching the Manager).

L0–L5 form a stable prefix → maximum prefix-cache hit rate.

6.2 Retrieval Algorithm

1. Daemon emits the query plus L4 + L5.

2. Server expands the query: a **HyDE** pass — Worker generates a hypothetical answer; we embed both query and hypothetical and average the vectors. This consistently lifts recall on code-heavy queries because user questions and code chunks live in different embedding regions.
3. Server returns the embedded query to the daemon.
4. **Daemon** runs the vector search against the **local LanceDB** → top-50 candidates (sim threshold 0.30, recall-optimized).
5. Daemon ships the 50 candidate chunks back to the server.
6. Server cross-encodes with BGE-reranker → top-10.
7. Filter by ownership rules (`secrets/` , `.env*` , `**/credentials*` , etc.).
8. Format with `path:lines [language]` headers so the Manager can cite locations. Pack into L6.

Vectors stay local (privacy + zero-roundtrip query); reranking happens server-side because the cross-encoder is non-trivial to ship to laptops.

6.3 Worker Briefing (the "scouring" step)

Before the Manager generates, the orchestrator dispatches parallel Worker tasks:

- "*Summarize these 3 files into ≤200 tokens each, preserving function signatures and side-effects.*"
- "*Triage this 8 KB pytest output; return failing test name, line number, and a one-sentence root-cause hypothesis.*"
- "*Compress this 12 KB* `debug.stack_trace` *result into the call chain only.*"

Worker output lands in L7 with provenance tags (`source: worker, basis: file=session.py`). The Manager reasons over distilled facts, not raw bytes.

---

7. Vector Database — Bundled, Invisible

7.1 User model: "Add Project"

The user never sees the vector DB. The entire UX is:

1. Click **+ Add Project**, pick a folder.
2. A status strip appears: *Indexing* `repo-name` *… 47 % (12,431 files)*.
3. The chat is usable immediately; retrieval falls back to lexical (ripgrep) until the index is warm enough.
4. Once green, the project is queryable. Switching between projects is a dropdown.
5. To remove a project: right-click → **Remove**. Storage is reclaimed.

That's it. No "vector database settings", no "embedding model" exposed. If the user is technical and goes looking, there's a developer panel in Settings that exposes index size and a "Rebuild" button — but it's hidden by default.

7.2 Where it lives

Inside the Electron app's `userData` directory. On Windows: `%APPDATA%/CompanyCode/projects/<project-id>/`. The path is managed by the Rust daemon and not exposed in the UI.

```
%APPDATA%/CompanyCode/
  projects/
    <project-id>/                          # UUIDv4
      meta.json                            # name, root path, model id, last index time
      lance/
        chunks/                            # vectors + chunk payload + metadata
        symbols/                           # file → exported symbol map
      cache/
        embeddings.bin                     # content-hash keyed embedding cache
      logs/
        index.log
```

`meta.json` records the embedding model id and dimension so we never mix incompatible vectors after a model upgrade.

7.3 Engine: LanceDB (Rust SDK)

LanceDB is Rust-native, runs in-process, has no sidecar, and the columnar Lance format spills to disk gracefully past 1 M vectors. Apache 2.0. The Rust SDK is first-class.

Other options were re-evaluated and rejected: Qdrant requires a separate process (extra installer surface); Chroma is weaker past 100K vectors; pgvector wants a DB server.

7.4 Indexing Strategy

On Add Project:

1. Daemon walks the worktree, respecting `.gitignore` and built-in default excludes (`node_modules/`, `vendor/`, `.venv/`, `dist/`, `target/`, `__pycache__/`, `*.lock`, `*.min.js`).
2. Per file:
   - Parse with **tree-sitter** (Rust crate `tree-sitter` + per-language grammars). Plaintext fallback for unsupported languages.
   - Chunk at AST-meaningful boundaries (cAST approach): one chunk per function / class / top-level block. Soft-split at 1500 tokens; never mid-statement.
   - Each chunk gets the leading docstring and ±2 lines of surrounding context so the embedding has scope.
3. Batch chunks (256/request) → POST to server's `/embed` endpoint.
4. Server returns 1024-dim vectors from `jina-embeddings-v3` (retrieval-document mode).
5. Daemon writes to LanceDB with metadata: `{ path, lang, symbol_kind, symbol_name, line_start, line_end, content_hash, indexed_at }`.

Performance: ~30–50 K LOC/min on a typical repo (network-bound on the embed call, not CPU-bound on chunking). A 500K-LOC repo indexes in ~10–17 min the first time.

Incremental updates: the `notify` Rust crate watches the worktree → debounce 500 ms → re-chunk only touched files. Embedding cache keyed by chunk content hash; unchanged chunks skip re-embed entirely. Typical small edit = re-embed 1–3 chunks.

7.5 Pollution avoidance

- **One LanceDB per project.** Never co-mingle. This is the single biggest cause of "RAG made it worse" in dev tools and we just don't do it.
- **Shared knowledge index** (company docs / ADRs / runbooks) lives separately on the server in **Qdrant**, read-only, ingested nightly from Confluence/SharePoint/wikis. The orchestrator queries it as a **distinct** retrieval pass and tags results `source: shared_docs` so the Manager can cite provenance.
- The user can implicitly enrich a project by dragging files into the chat ("attach"); attached files are indexed into the same project DB with `source: user_attached` tags. There are no other knobs to twiddle.

7.6 Index hygiene

- Stale eviction: any chunk whose source file was deleted >7 days ago is purged on the next compaction.
- LanceDB compaction is triggered on app startup if fragmentation > 30%.
- **Embedding-model upgrades** are atomic per project: dual-index in the background, then swap. No re-index storm.

---

8. Tools

The Manager must *act*. Tool calls are first-class in the protocol.

8.1 Tool surface (executed in the Rust daemon, sandboxed)

Tool	Purpose	Sandbox	Confirmation
`shell.exec`	Run a shell command	`cwd` = project root; outbound network blocked unless allowlisted; 30 s timeout	**Yes** before execution, with command preview
`fs.read`	Read a file	Within project	No
`fs.write`	Propose a file edit	Within project; **always dry-run** — produces a hunk into the review surface, never writes directly	Approval is the review-loop "Apply"
`git.status` / `git.diff` / `git.log` / `git.blame`	Read-only git introspection	Read-only	No
`vector.search`	Re-query the local index with a new term	Local	No
`lsp.symbols` / `lsp.refs` / `lsp.diagnostics` / `lsp.hover`	Language server queries	Local	No
`web.search`	Hit an internal docs endpoint	Allowlisted domains only	Configurable per-project
`debug.*`	**DAP debugger** (see §8.2)	Sandboxed subprocess	**Yes** on session start; subsequent steps within the same session are pre-approved

8.2 Debugger Tool — DAP Integration

This is one of the highest-leverage capabilities in the system. Rather than guessing what a variable holds at runtime, the agent **looks**.

8.2.1 Why DAP

The Debug Adapter Protocol is the same wire format VS Code uses for every debugger it supports. It's stable, JSON-RPC, and there's a mature debug adapter for nearly every language: `debugpy` (Python), `vscode-js-debug` (Node/Chrome), `codelldb` (Rust/C/C++), `delve` (Go), Java DI, `netcoredbg` (.NET), `rdbg` (Ruby).

By targeting DAP we get every language for free, and the same agent tool surface works across all of them.

8.2.2 Tool surface

Tool	Args	Returns
`debug.start`	`{ lang, program, args, env, cwd, just_my_code? }`	`session_id`
`debug.set_breakpoint`	`{ session_id, file, line, condition?, hit_condition?, log_message? }`	`bp_id`, `verified`
`debug.list_breakpoints`	`{ session_id }`	`[{file, line, hit_count, …}]`
`debug.remove_breakpoint`	`{ session_id, bp_id }`	`ok`
`debug.continue` / `step_over` / `step_into` / `step_out`	`{ session_id, thread_id? }`	streams `tool_event` on stop
`debug.stack_trace`	`{ session_id, thread_id }`	`[{frame_id, name, file, line}]`
`debug.scopes_and_variables`	`{ session_id, frame_id, max_depth? }`	structured tree
`debug.evaluate`	`{ session_id, frame_id, expr, context: "watch" | "repl" }`	`{ result, type }`
`debug.terminate`	`{ session_id }`	`ok`

`debug.set_breakpoint` supports **conditional** and **hit-count** breakpoints, plus **logpoints** (a breakpoint that prints rather than stops — incredibly useful for the agent to instrument code without modifying it).

8.2.3 Implementation in the Rust daemon

```
┌──────────────── Rust daemon ─────────────────┐
│                                              │
│  Debugger Manager                            │
│   ├─ session map: session_id → DapClient     │
│   ├─ launches debug-adapter subprocess       │
│   │     stdio JSON-RPC ↔ adapter             │
│   ├─ translates agent tool calls →           │
│   │     DAP requests                         │
│   └─ multiplexes async DAP events →          │
│         tool_event messages upstream         │
│                                              │
│  Subprocesses (one per active session):      │
│   • python -m debugpy.adapter                │
│   • node ./js-debug/src/dapDebugServer.js    │
│   • codelldb                                 │
│   • dlv dap                                  │
└──────────────────────────────────────────────┘
```

Rust crates for this: `tokio` (process + IO), `serde_json` (DAP messages), and a thin DAP client we own (~600 lines of Rust; the protocol surface we use is small — Initialize, Launch/Attach, SetBreakpoints, Continue, StackTrace, Scopes, Variables, Evaluate, Terminated).

8.2.4 Auto-detected launch configurations

The daemon looks for the obvious entry points so the agent doesn't have to invent them:

- Python: a `pytest <test_file>::<test_name>`-shaped launch when the user's task references a failing test; otherwise read `pyproject.toml` / `setup.cfg`.
- Node: `package.json#scripts.test` or a `node <file>` if the agent is debugging a script.
- Rust: `cargo test <name> -- --nocapture --test-threads=1` then attach `codelldb`.
- Go: `go test -run <name>` then attach `dlv`.
- For everything else, the agent calls `shell.exec` to print the project's test runner config and synthesizes a launch JSON from there.

8.2.5 Confirmation UX

The first `debug.start` of a session pops a modal: "*Allow CompanyCode to launch* `pytest auth/session_test.py -k test_session_expires` *and attach a debugger?*" — with the full command, env diff, and cwd visible. After approval, **all subsequent** `debug.*` **calls in that session are pre-approved** (the agent can step, set breakpoints, evaluate without further prompts). When the session terminates, approval is gone; a new session re-prompts.

8.2.6 Why this is valuable

For a class of bugs ("my function returns the wrong value", "this test fails for a non-obvious reason", "intermittent failure"), reading code is *not enough*. The agent needs runtime ground truth. Setting a breakpoint at the suspected line, running the failing path, and inspecting the actual variable values converts a research problem into a measurement problem. Empirically this is one of the largest accuracy lifts available — and it's also a feature the developer can directly observe and verify.

8.3 Tool result handling

Tool output flows back as `tool_result`. Anything >2000 tokens is intercepted by the orchestrator and routed to the Worker for compression first, with a hard system-prompt rule that the compressed form must preserve filenames, line numbers, error messages, and variable names verbatim. Only the compressed form reaches the Manager's L7.

`tool_event` messages (e.g., debugger stopped) are similarly compressed before being dropped into L7, but the agent can request the raw form with `debug.scopes_and_variables` when needed.

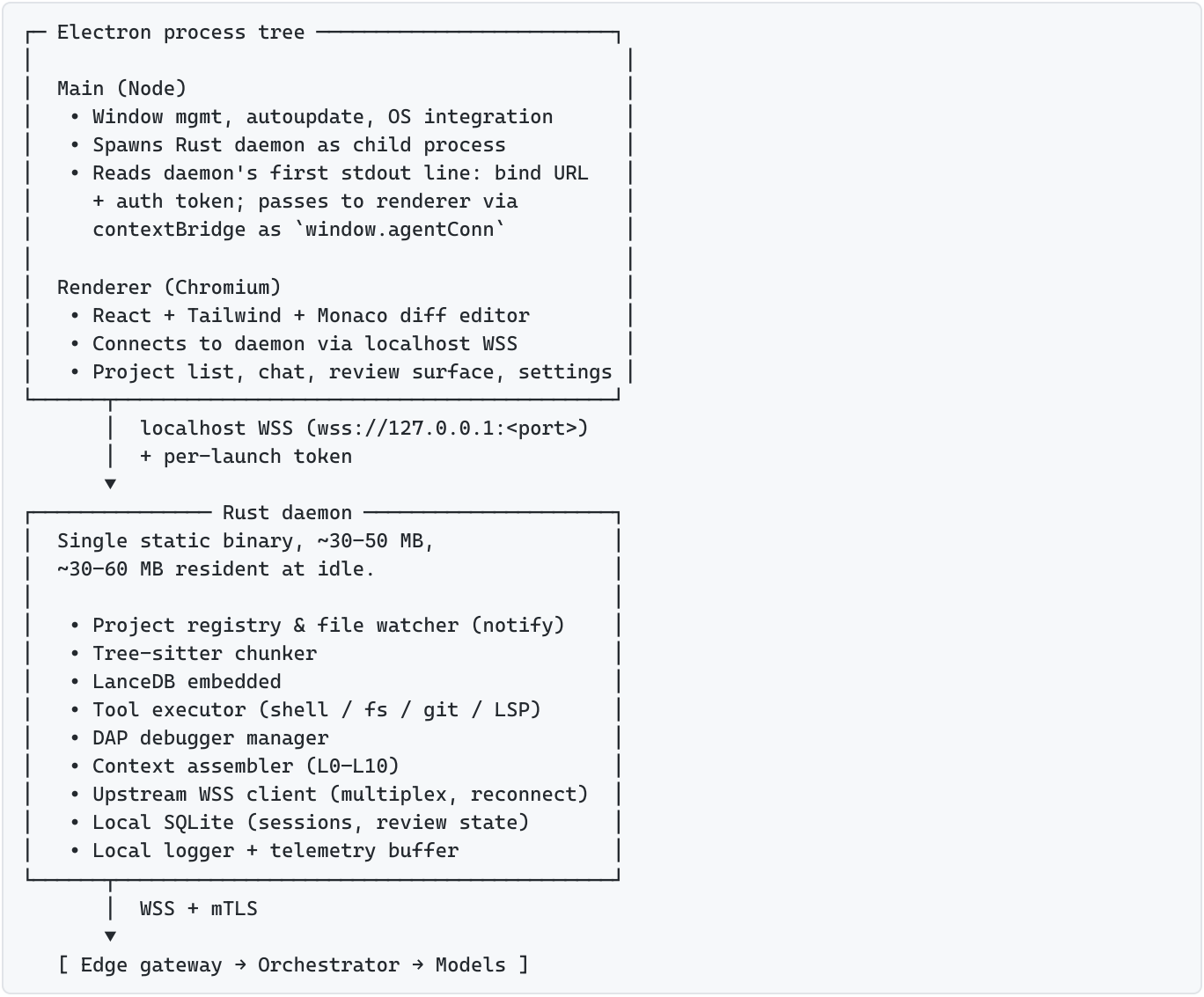
---

9. Client Application — Electron

9.1 Why Electron

We're building one focused product surface, on every platform our developers use. Electron lets us ship the React + Monaco diff view we want, with one codebase, with zero learning curve for any web-fluent contributor. The well-known cost — memory — is mitigated by pushing every heavy operation out of Electron and into the Rust daemon (§9.2). The Electron renderer holds **only** the UI; it never holds LanceDB, never parses ASTs, never executes tools.

9.2 Process model



9.3 Why the daemon is in Rust

Concern	Why Rust wins
Memory footprint	Single static binary at 30–60 MB resident; Node would sit at ~150 MB with `node_modules` reachable
LanceDB bindings	LanceDB is itself written in Rust; using the native SDK avoids the FFI cost the Node binding pays
Tree-sitter	The reference `tree-sitter` library is C with a Rust crate that's the cleanest binding; grammar loading is faster
File watching	`notify` is the most reliable cross-platform watcher in any language
Concurrency	`tokio` + `axum` for IPC, `tokio-tungstenite` for the upstream WSS, all under one async runtime; manageable at high tool-call concurrency
Long-lived correctness	No GC pauses during a 30-min agentic run; deterministic shutdown so debugger subprocesses always get cleaned up
Distribution	One static binary, no runtime dependencies, no `node_modules` to bundle

9.4 Crate map (daemon)

```
[dependencies]
tokio              = { version = "1", features = ["full"] }
tokio-tungstenite  = { version = "0.24", features = ["rustls-tls-native-roots"] }
axum               = "0.7"        # localhost IPC server
serde              = { version = "1", features = ["derive"] }
serde_json         = "1"
rmp-serde          = "1"          # MessagePack for binary mode
rustls             = "0.23"
notify             = "7"          # filesystem watcher
tree-sitter        = "0.24"
tree-sitter-rust   = "0.24"       # plus per-language grammars
lancedb            = "0.10"       # vector DB
sqlx               = { version = "0.8", features = ["sqlite","runtime-tokio-rustls"] }
ulid               = "1"
tracing            = "0.1"
tracing-subscriber = "0.3"
keyring            = "3"          # OS keychain for at-rest encryption key
gitleaks-rs        = "*"          # or shell out to gitleaks for secret scrubbing
```

9.5 IPC: localhost WSS, not Electron contextBridge

The renderer talks to the daemon over loopback WebSocket on a randomly-allocated high port, with a per-launch HMAC token in the upgrade headers. Reasons:

- The renderer can speak to the daemon directly without round-tripping through the Electron main process — same protocol surface as the upstream WSS, so the daemon has *one* protocol implementation.

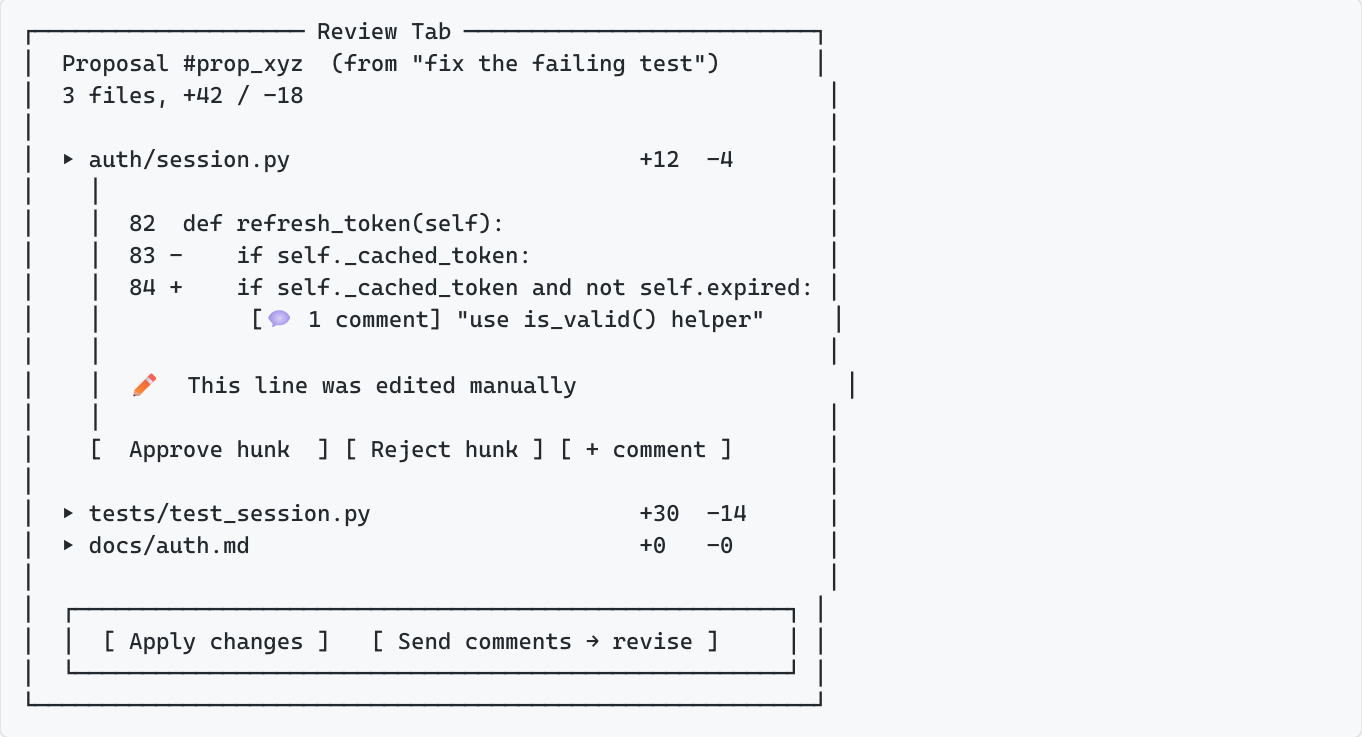
- The daemon is then portable: a CLI (`companycode`) and any future client can speak the same loopback protocol.
- The daemon survives Electron restarts cleanly; the renderer just reconnects.

---

10. Iterative Review Loop

This is the primary interaction surface. After the agent emits a multi-file diff, the user reviews it in the app, may comment on specific lines, may edit directly, and either applies or sends comments back for revision.

10.1 The review surface



10.2 Per-line affordances

- **Click a line** in the diff → comment input appears anchored to that line (file + line + hunk_id).
- **Direct edit in the diff** → tracked as a manual delta; the diff viewer keeps both "agent's version" and "what the user typed" so we can show the agent exactly what changed.
- **Approve hunk** / **Reject hunk** — coarse signals that flow into preference data.
- The review surface is editable indefinitely; the user can think, walk away, come back.

10.3 The two terminal actions

- **Apply changes** — daemon writes the agreed-upon hunks to disk (or applies them as a `git apply` patch). The proposal is closed; review state for that proposal is cleared from L9.

- **Send comments → revise** — the next request is constructed with the review state injected as L9; the agent reasons about the proposal it just made and the user's feedback, and emits a new proposal.

10.4 Review state schema (what L9 contains on revise)

```
{
  "previous_proposal_id": "prop_xyz",
  "previous_proposal_summary": "<200 tokens, distilled by Worker>",
  "comments": [
    { "file": "auth/session.py", "line": 84,
      "hunk_id": "h_2",
      "text": "use the is_valid() helper instead of inlining" },
    { "file": "tests/test_session.py", "line": 31,
      "hunk_id": "h_5",
      "text": "this assertion should also check session.user_id" }
  ],
  "manual_edits": [
    { "file": "auth/session.py",
      "delta_against_proposal": "@@ ... @@\n-    if self._cached_token and not self.expired:\n+
if self._cached_token and self.is_valid():" }
  ],
  "approved_hunks": ["h_1", "h_3"],
  "rejected_hunks": ["h_4"],
  "instruction": "Revise the proposal. Preserve manual edits unless they conflict with the
comments. Address every comment. Keep approved hunks unchanged."
}
```

The Worker compresses `previous_proposal_summary` from the full proposal so it fits in 200 tokens. Approved/rejected hunks help the Manager understand what's already settled and shouldn't be re-litigated.

10.5 Why this works

- **Locality.** Comments are anchored to lines, not to "the diff" at large. The agent gets unambiguous targeting.
- **Two channels of signal.** Comments tell the agent *what to think differently*; manual edits tell the agent *exactly what the user wanted instead*. The combination is far stronger than either alone.
- **No context blowup.** The previous proposal is summarized before re-injection; only the *deltas* (comments, manual edits) and the structural decisions (approved/rejected hunks) flow through verbatim.
- **Preference signal for free.** Every approve/reject/comment/edit becomes a labeled data point for the DPO loop (§12). The user is producing training data by doing the review they were going to do anyway.

10.6 Loop bounds

Practical limits we enforce:

- A single proposal can iterate up to 5 times before the agent is forced to summarize and start a fresh proposal (prevents context bloat).

- Manual edits are diffed against the original proposal *and* against the file on disk; if they diverge, the user is alerted before the next revision so we don't silently overwrite their work.
- "Apply" requires git working tree to be clean of conflicts in the touched files; otherwise the user is prompted to commit/stash first.

---

11. Security & Privacy

11.1 Data flow boundaries

Stays on the laptop	Crosses to the server (request-time)	Stays on the server
Full source files	L0–L10 prompt layers (the slice we deliberately retrieved)	Model weights, KV cache
Local vector DB	Retrieved chunk payloads	Structured logs (content-hash redacted)
Indexing intermediate state	Tool outputs (compressed by Worker on server, then injected)	Preference DB
Tool execution & raw output	Telemetry (opt-in, scrubbed)	Shared knowledge index
Debugger sessions and inspected variables	Compressed summaries only	

Variable values inspected by the debugger are **not** uploaded raw. The Worker compresses them server-side from a daemon-supplied summary, and the raw values stay on the laptop.

11.2 Trust boundaries

- **mTLS** on the upstream WebSocket — both ends authenticated by certificate.
- **JWT** carries developer identity (15-min TTL); orchestrator stamps this on every log entry.
- Tool execution requires user confirmation by default; debugger sessions follow the per-session approval model in §8.2.5.
- **Prompt-injection mitigation:** tool output is wrapped in a `<tool_result untrusted="true">` boundary; the system prompt instructs the Manager to treat its content as data, not instructions.
- The localhost WSS between renderer and daemon is bound to 127.0.0.1, has a per-launch HMAC token, and refuses non-loopback connections.

11.3 Secrets hygiene

- Pre-index scrub: `gitleaks` rule packs run during chunking. Chunks with apparent secrets are excluded entirely (never embedded, never stored).

- Egress filter on the daemon: outbound network only to the configured server FQDN.
- Local DB encrypted at rest using a key derived from the OS keychain (Keychain on macOS, DPAPI on Windows, libsecret on Linux). The `keyring` Rust crate handles all three.
- Debugger logs and `tool_event` payloads are written to disk only with content-hash redaction by default.

---

12. Feedback Loop & Continuous Improvement

12.1 Signals captured

The review loop (§10) is the feedback engine. Every proposal generates rich signal:

- **Hunk-level approve/reject** — direct preference data.
- **Per-line comments** — natural-language critique, paired with the hunk they targeted.
- **Manual edits** — gold-standard "what the user actually wanted" data, paired with the agent's proposal.
- Plus the usual: prompt fingerprint, retrieved chunk IDs, worker briefs, tools called, latencies, tokens in/out.

Stored locally first (in the daemon's SQLite). Periodically synced to a server-side **Postgres** `feedback` DB — opt-in per developer, with deterministic-regex + small-classifier scrubbing for PII and secrets pre-upload.

12.2 Preference dataset construction

For each proposal that ended in **Apply**:

- Each approved hunk → `(prompt, agent_output, user_action=approved)`.
- Each manually-edited hunk → preference pair `(agent_output, user_edit)` with `user_edit` as chosen.
- Each rejected hunk → preference pair `(agent_output, regenerated_output)` with the eventually-applied output as chosen.
- Each comment → `(prompt, comment_text)` becomes a synthetic SFT pair (the Manager learns to anticipate this kind of feedback).

Plus synthetic negatives: take an applied hunk, programmatically introduce a known regression (delete an import, break a test), pair as rejected.

Aim: ~10–30 K preference pairs per quarter at 10 devs; scales with usage.

12.3 Training cadence

- Quarterly DPO fine-tune on the frozen Qwen3.6-27B base → produces a **LoRA adapter** (~200 MB), not a new full checkpoint.
- vLLM hot-loads the adapter; deploy is an adapter-id flip in the orchestrator config — no model reload, no downtime.
- A/B rollout: 10% canary → 50% → 100% over two weeks, gated by win-rate against a frozen "canonical company tasks" eval set.

12.4 Why DPO

- No reward model to train and align.
- The accept/reject/edit-then-accept signal is exactly the binary preference shape DPO consumes. Zero data-shape work.
- Computationally cheap and stable; effective at the thousands-of-pairs scale we'll have.
- LoRA-shaped output plugs into vLLM's multi-LoRA serving without redeploy.

12.5 Guardrails

- Pre-upload scrub: `gitleaks` rules + a small classifier flag any prompt or output containing apparent secrets/PII; flagged records are dropped at the source.
- Hold-out gate: if win-rate against the canonical set drops vs. the previous adapter, deployment is blocked automatically.
- Manual review queue for low-confidence preference pairs (close margins, small samples).
- Per-dev opt-out stored in the daemon config; respected at the source so opted-out telemetry never leaves the laptop.

---

13. Observability

13.1 Server-side

- **Prometheus:** per-tier QPS, TTFT, decode tok/s, KV-cache utilization, prefix-cache hit rate, queue depth, GPU memory used.
- **Loki:** structured logs with prompt fingerprint and metadata; raw prompts not logged by default (opt-in per-team, with strict retention).
- **Tempo / OpenTelemetry:** spans for `request → orchestrator → worker → tool`. Trace IDs surfaced in the client UI on error so devs can link a bug report directly to a trace.
- **Grafana** dashboards for: model utilization, queue depth, tool-call success rate, retrieval recall@10, **review-revise rate** (proposals applied vs. revised vs. abandoned).

13.2 Client-side

- Local rolling log buffer (last 24 h on disk, capped at 100 MB).
- Health panel in the UI: WS connection status, indexing percentage, last successful sync, daemon version.
- Crash reports via Sentry — opt-in, with PII scrubbed.

---

14. Phased Rollout

Phase	Weeks	Scope
0 — Spike	1–2	Single dev laptop, one project, Manager only, no RAG. Prove end-to-end: WSS, prompt assembly, streaming, one tool call, one diff returned to a stub UI.
1 — Foundation	3–6	Wire protocol locked. Electron shell with project list. Rust daemon: file watcher, LanceDB, tree-sitter chunker, upstream WSS.
2 — Manager + Worker	7–9	vLLM on 2× H100 with Qwen3.6-27B + Qwen3-Coder-7B. jina-v3 embed, BGE-reranker. Retrieval pipeline live end-to-end.
3 — Tool surface	10–12	shell / fs / git / LSP tools with confirmation UI. Prompt-injection defense.
4 — Debugger	13–15	DAP integration: Python first (debugpy), then Node, then Rust/Go. Per-session approval flow.
5 — Review loop	16–18	Monaco diff viewer. Comment threading. Manual-edit tracking. L9 review-state injection.
6 — Pilot	19–22	Roll to 10 internal devs on their own projects. Wire feedback capture.
7 — Hardening	23–26	mTLS, secret scrubbing, egress controls, observability dashboards, on-call runbook.
8 — DPO	27–30	First DPO LoRA on collected review signal; canary deploy; canonical-eval gate.
9 — GA	31+	Wider rollout. Evaluate scaling to 20–50 devs (§5.5). Shared knowledge index ingest pipeline.

15. Open Questions / Decisions to Validate

1. **Conditional breakpoints latency.** DAP `set_breakpoint` with a complex condition can be slow on some adapters (debugpy especially). May need a daemon-side cap so the agent doesn't set a 200ms-evaluating breakpoint inside a tight loop.
2. **Worker dispatch policy.** Start with the rules in §5.6. Replace with a learned classifier once we have ≥50 K traces.
3. **Shared knowledge index granularity.** Per-team Qdrant collections vs. one global with team metadata? Lean per-team to limit blast radius of a bad ingest.
4. **Multi-repo / monorepo workspaces.** Should "Project" map 1:1 to a worktree, or allow multiple worktrees per project? Lean 1:1 for v1; revisit if monorepo users complain.
5. **Embedding-model upgrade path.** Re-embedding the entire fleet's projects has real cost. Define the upgrade policy: keep the previous model live for a quarter, dual-index in the background, swap atomically per project.

6. **DAP sandboxing on Windows.** Some adapters expect a fully-fledged shell environment; we need to confirm `debugpy` and `vscode-js-debug` work cleanly under our process-confined launch.
7. **Review revision limit (§10.6).** Five iterations is a guess. Tune from pilot data.

Appendix A — Tech Stack Summary

Layer	Choice	Rationale
Manager model	**Qwen3.6-27B-FP8** (dense)	Strongest open agentic coder in 2026 (77.2 SWE-bench Verified, 59.3 Terminal-Bench 2.0). Dense 27B beats sparse 35B-A3B and 397B MoE on agentic benchmarks.
Worker model	**Qwen3-Coder-7B-FP8**	Fast, accurate compression/summarization; same tokenizer family as Manager
Embedding	jina-embeddings-v3 (1024-dim, 8K ctx)	Long-doc strength, task-specific LoRA modes, retrieval-mode adapter
Reranker	BGE-reranker-v2-m3 (FP16)	High-accuracy cross-encoder, fits a slice of one GPU
Inference engine	vLLM	Continuous batching + prefix cache + multi-LoRA + MTP speculative decoding (Qwen 3.6 recipe)
GPU baseline	**1× RTX PRO 6000 Blackwell 96GB** (~$8.5K)	10 concurrent agentic devs with headroom; +1 Pro 6000 per +12 devs. H100 only worth it past ~50 devs when NVLink-based TP starts paying for itself (§5.1, §5.5)
Local vector DB	**LanceDB (Rust SDK, embedded)**	No sidecar, columnar, scales past 1M vectors, native Rust
Shared docs DB	Qdrant (server-side, read-only)	Excellent filtered search, mature ops, isolated from per-project indices
AST chunker	tree-sitter (cAST style)	150+ languages, syntactic boundaries
Debugger	DAP via subprocess adapters (debugpy, vscode-js-debug, codelldb, dlv, …)	Standard protocol; one tool surface for every language
Transport	WSS + mTLS, JSON-lines (MessagePack opt)	Streaming, multiplexed, debuggable, proxy-friendly
Auth	OIDC SSO → JWT (15-min TTL)	Standard enterprise auth
Client UI	**Electron + React + Monaco diff editor**	Web-tech velocity for the review surface; daemon split keeps the renderer small
Local agent daemon	**Rust** (tokio + axum + lancedb + tree-sitter + notify)	Single static binary, low memory, native LanceDB & tree-sitter, deterministic shutdown for debugger subprocesses
Feedback DB	Postgres + S3 (cold archive)	Standard, cheap, queryable
Fine-tuning	DPO via LoRA, quarterly	Cheap, stable; plugs straight into review-loop signal

Layer	Choice	Rationale
Telemetry	Prometheus + Loki + Grafana + OTel/Tempo	Standard stack

---

Appendix B — Sources

- Qwen3.6-27B on Hugging Face
- Qwen3.6-27B-FP8 on Hugging Face
- Qwen3.6-27B: Flagship-Level Coding in a 27B Dense Model — Qwen blog
- Alibaba Qwen Releases Qwen3.6-27B — MarkTechPost
- Qwen3.5 & Qwen3.6 Usage Guide — vLLM Recipes
- Qwen3.6 — How to Run Locally — Unsloth
- Best Embedding Models 2026 — MTEB
- Vector Database Benchmarks 2026 — CallSphere
- LanceDB — GitHub
- vLLM Automatic Prefix Caching docs
- cAST: AST-Structural Code Chunking — arXiv
- Direct Preference Optimization — arXiv 2305.18290
- Debug Adapter Protocol — Microsoft spec
- mcp-debugger — DAP for AI agents
- A Technical Blueprint for Local-First AI with Rust and Tauri — Musa Bello
- NVIDIA RTX PRO 6000 Blackwell Pricing (April 2026) — Thunder Compute
- Comparing the NVIDIA RTX Pro 6000 vs. H100 for AI Inference — OpenMetal
- RTX PRO 6000 Benchmarks: 30B AWQ, 70B FP8, and Cost Per Million Tokens — Spheron
- RTX 5090 LLM Benchmarks: Cost Per Million Tokens Deep Dive — Spheron
- RTX PRO 6000 Blackwell vs H100 vs RTX 5090 — VRLA Tech